

Подсказки и типовые ошибки

Когда вы впервые запускаете rust-gdb, perf или flamegraph, почти всегда что-то идёт не так: исходники не отображаются, точки останова не срабатывают, а профилировщик показывает только адреса вместо имён функций.

❶ Ошибка 1. Отладчик не показывает исходный код

При запуске rust-gdb вы видите только машинный код или адреса, но не строки Rust-кода.

Причина: Бинарь собран без debug-символов (DWARF) или они были удалены.

Что сделать:

- Убедитесь, что используете debug-профиль: `cargo build`, а не `cargo build --release`.
- Проверьте наличие debug-секций: `readelf -S target/debug/your_app | grep debug`.
- Если секций нет — посмотрите Cargo.toml: возможно, там вручную отключена генерация debug-информации.

❷ Ошибка 2. Breakpoint не срабатывает

Вы ставите breakpoint, но программа не останавливается.

Возможные причины:

- Breakpoint поставлен на строку, которая была оптимизирована компилятором (release-профиль).
- Функция вообще не вызывается.
- Breakpoint поставлен на строку с макросом, а реальный код после развёртки — в другом месте.

Что сделать:

- Отлаживать в debug-профиле.
- Ставить breakpoint по имени функции: `break function_name`.
- Убедиться, что функция реально выполняется, — банальный `println!` перед вызовом.

❸ Ошибка 3. Переменные показывают «мусор» или optimized out

В отладчике переменные имеют странные значения или помечены как optimized out.

Причина: Компилятор оптимизировал код, и переменная была удалена/переложена, отладчик не может восстановить её состояние.

Что сделать:

- Использовать debug-профиль — `cargo build` без `--release`.
- Если критично дебажить release-код — точно ослабить оптимизацию:
 - добавить `#[inline(never)]` к нужной функции;
 - подстроить профиль в Cargo.toml, чтобы не выносить всё подчистую.

❹ Ошибка 4. Профилировщик показывает только адреса, а не имена функций

В perf report или flamegraph вы видите только шестнадцатеричные адреса.

Причина: Бинарь собран без debug-символов или от них сделали strip.

Что сделать:

В Cargo.toml для release-профиля:

```
[profile.release]
debug = 1 # или debug = 2 для максимальной детализации
```

Пересобрать: `cargo build --release`.

Если есть отдельные debug-файлы — убедиться, что профилировщик знает, где их искать.

❺ Ошибка 5. Стеки вызовов неполные или рваные

В flamegraph стеки обрываются, какие-то функции пропадают.

Причина: Агрессивные оптимизации (inlining, удаление frame pointers) мешают нормально восстановить стек.

Что сделать:

Включить сохранение frame pointers:

```
RUSTFLAGS="-C force-frame-pointers=yes" cargo build --release
```

Использовать свежий perf и сбор по DWARF (--call-graph dwarf), если нужно.

❻ Ошибка 6. Профилировщик сильно замедляет программу

С профилировщиком программа работает в 10–100 раз медленнее.

Причина: Вы используете тяжёлое инструментальное профилирование (instrumentation) или слишком частый sampling.

Что сделать:

- Для общего анализа использовать sampling (perf record, cargo flamegraph), а не тотальное инструментирование.
- Уменьшить частоту сэмпинга (например, perf record -F 100 вместо нескольких тысяч Гц).
- В проде включать детализацию точно (метрики/трейсинг), а не вокруг всего сервиса.